```sql
-- Q1. How many unique post types are found in fact_content?
SELECT
    COUNT(DISTINCT post_type) AS unique_post_type_count,
    GROUP_CONCAT(DISTINCT post_type ORDER BY post_type SEPARATOR ', ') AS post_type_names
FROM fact_content;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

unique_post_type_count	post_type_names
4	IG Carousel, IG Image, IG Reel, IG Video

```sql
-- Q2. What are the highest and lowest recorded impressions for each post type?
SELECT
    post_type,
    MAX(impressions) AS highest_impressions,
    MIN(impressions) AS lowest_impressions
FROM fact_content
GROUP BY post_type
ORDER BY post_type;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

post_type	highest_impressions	lowest_impressions
IG Carousel	9677	3264
IG Image	129694	23367
IG Reel	339708	87570
IG Video	73321	8741

```sql
-- Q3. Filter weekend posts published in March and April
SELECT
    fc.*,
    dd.month_name,
    dd.weekday_name,
    dd.weekday_or_weekend
FROM fact_content AS fc
JOIN dim_dates AS dd
    ON fc.date = dd.date
WHERE dd.month_name IN ('March', 'April')
  AND dd.weekday_or_weekend = 'Weekend'
ORDER BY fc.date;
```

	A	B	C	D	E	F	G	H	I	J
	date	post_category	post_type	video_duration	carousel_	impressions	reach	shares	follows	likes
	04-03-2023	Earphone	IG Video	291	0	12265	3668	69	92	327
	05-03-2023	Smartwatch	IG Image	0	0	62770	18001	273	360	1194
	11-03-2023	Mobile	IG Carousel	0	3	5899	1093	45	12	53
	12-03-2023	Laptop	IG Image	0	0	79416	23474	327	259	1235
	18-03-2023	Mobile	IG Carousel	0	3	9157	2254	67	58	55
	19-03-2023	Smartwatch	IG Carousel	0	3	4146	1079	42	17	43
	25-03-2023	Earphone	IG Reel	22	0	132284	66721	1093	1482	3622
	26-03-2023	Mobile	IG Image	0	0	63425	26113	435	336	1994
	01-04-2023	Mobile	IG Carousel	0	3	4549	1052	27	18	35
	02-04-2023	Earphone	IG Video	163	0	54672	16126	172	182	938
	08-04-2023	Other Gadgets	IG Video	258	0	37955	12663	204	164	753
	09-04-2023	Mobile	IG Image	0	0	52278	14438	271	167	1393
	15-04-2023	Laptop	IG Reel	30	0	123270	39850	296	1486	3926
	16-04-2023	Other Gadgets	IG Reel	29	0	115701	66829	937	929	5749
	22-04-2023	Laptop	IG Video	172	0	33604	14682	255	349	1038
	23-04-2023	Earphone	IG Video	229	0	36973	13629	224	244	929
	29-04-2023	Earphone	IG Video	206	0	43526	11799	134	138	646
	30-04-2023	Mobile	IG Reel	59	0	185017	63990	1010	2238	6039

```sql
/* 4. Create a report to get the statistics for the account. The final output
includes the following fields:
• month_name
• total_profile_visits
• total_new_followers  */
SELECT
    dd.month_name,
    SUM(fa.profile_visits) AS total_profile_visits,
    SUM(fa.new_followers) AS total_new_followers
FROM fact_account AS fa
JOIN dim_dates AS dd
    ON fa.date = dd.date
GROUP BY MONTH(dd.date), dd.month_name
ORDER BY MONTH(dd.date);
```

Result Grid | Filter Rows: | Export: | W

	month_name	total_profile_visits	total_new_followers
▶	January	26512	17053
	February	20628	15254
	March	23132	18285
	April	29852	21799
	May	106571	66984
	June	103350	76942
	July	54352	33302
	August	42094	24371
	September	41522	28523

```sql
/* 5. Write a CTE that calculates the total number of 'likes' for each
'post_category' during the month of 'July' and subsequently, arrange the
'post_category' values in descending order according to their total likes. */

WITH july_likes AS (
    SELECT
        fc.post_category,
        SUM(fc.likes) AS total_likes
    FROM fact_content AS fc
    JOIN dim_dates AS dd
        ON fc.date = dd.date
    WHERE dd.month_name = 'July'
    GROUP BY fc.post_category
)
SELECT
    post_category,
    total_likes
FROM july_likes
ORDER BY total_likes DESC;
```

Result Grid | Filter Rows:

post_category	total_likes
Other Gadgets	26519
Tech Tips	20296
Mobile	16338
Earphone	14435
Smartwatch	3918

```
/* 6. Create a report that displays the unique post_category names alongside
their respective counts for each month. The output should have three
columns:
• month_name
• post_category_names
• post_category_count
Example:
• 'April', 'Earphone,Laptop,Mobile,Other Gadgets,Smartwatch', '5'
• 'February', 'Earphone,Laptop,Mobile,Smartwatch', '4 */
```

```
SELECT
    dd.month_name,
    GROUP_CONCAT(
        DISTINCT fc.post_category
        ORDER BY fc.post_category
        SEPARATOR ','
    ) AS post_category_names,
    COUNT(DISTINCT fc.post_category) AS post_category_count
FROM fact_content AS fc
JOIN dim_dates AS dd
    ON fc.date = dd.date
GROUP BY MONTH(dd.date), dd.month_name
ORDER BY MONTH(dd.date);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

month_name	post_category_names	post_category_count
January	Earphone,Mobile,Smartwatch	3
February	Earphone,Laptop,Mobile,Smartwatch	4
March	Earphone,Laptop,Mobile,Smartwatch	4
April	Earphone,Laptop,Mobile,Other Gadgets,S...	5
May	Earphone,Laptop,Mobile,Other Gadgets,S...	6
June	Mobile,Other Gadgets,Smartwatch,Tech ...	4
July	Earphone,Mobile,Other Gadgets,Smartwa...	5
August	Earphone,Mobile,Other Gadgets,Smartwa...	5
September	Mobile,Other Gadgets,Smartwatch,Tech ...	4

```
/* 7. What is the percentage breakdown of total reach by post type? The final
output includes the following fields:
• post_type
• total_reach
• reach_percentage */
```

```sql
SELECT
    post_type,
    SUM(reach) AS total_reach,
    ROUND(
        100.0 * SUM(reach) /
        NULLIF((SELECT SUM(reach) FROM fact_content), 0),
        2
    ) AS reach_percentage
FROM fact_content
GROUP BY post_type
ORDER BY total_reach DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

post_type	total_reach	reach_percentage
IG Reel	5379091	61.63
IG Image	1866381	21.38
IG Video	1422300	16.30
IG Carousel	60465	0.69

```
/* 8. Create a report that includes the quarter, total comments, and total
saves recorded for each post category. Assign the following quarter
groupings:
(January, February, March) → “Q1”
(April, May, June) → “Q2”
(July, August, September) → “Q3”
The final output columns should consist of:
• post_category
• quarter
• total_comments
• total_saves */
```

```sql
SELECT
    fc.post_category,
    CASE
        WHEN MONTH(fc.date) BETWEEN 1 AND 3 THEN 'Q1'
        WHEN MONTH(fc.date) BETWEEN 4 AND 6 THEN 'Q2'
        WHEN MONTH(fc.date) BETWEEN 7 AND 9 THEN 'Q3'
    END AS quarter,
    SUM(fc.comments) AS total_comments,
    SUM(fc.saves) AS total_saves
FROM fact_content AS fc
WHERE MONTH(fc.date) BETWEEN 1 AND 9
GROUP BY
    fc.post_category,
    CASE
        WHEN MONTH(fc.date) BETWEEN 1 AND 3 THEN 'Q1'
        WHEN MONTH(fc.date) BETWEEN 4 AND 6 THEN 'Q2'
        WHEN MONTH(fc.date) BETWEEN 7 AND 9 THEN 'Q3'
    END
ORDER BY fc.post_category, quarter;
```

Result Grid | Filter Rows: | Export: | Wrap Cell

post_category	quarter	total_comments	total_saves
Earphone	Q1	351	2230
Earphone	Q2	589	3602
Earphone	Q3	427	3247
Laptop	Q1	418	2837
Laptop	Q2	452	2248
Mobile	Q1	1836	9843
Mobile	Q2	2313	17207
Mobile	Q3	1134	5285
Other Gadgets	Q2	1622	12041
Other Gadgets	Q3	964	4457
Smartwatch	Q1	600	2860
Smartwatch	Q2	1358	12581
Smartwatch	Q3	971	3326
Tech Tips	Q2	2201	17649
Tech Tips	Q3	1596	12976

```sql
/* 9. List the top three dates in each month with the highest number of new
followers. The final output should include the following columns:
• month
• date
• new_followers */

WITH ranked_dates AS (
    SELECT
        dd.month_name AS month,
        MONTH(fa.date) AS month_num,
        fa.date,
        fa.new_followers,
        ROW_NUMBER() OVER (
            PARTITION BY MONTH(fa.date)
            ORDER BY fa.new_followers DESC, fa.date ASC
        ) AS date_rank
    FROM fact_account AS fa
    JOIN dim_dates AS dd
        ON fa.date = dd.date
)

  SELECT
      month,
      date,
      new_followers
  FROM ranked_dates
  WHERE date_rank <= 3
  ORDER BY month_num, date_rank;
```

Result Grid | Filter Rows: | Export:

month	date	new_followers
January	2023-01-30	3186
January	2023-01-03	2959
January	2023-01-23	1003
February	2023-02-01	4106
February	2023-02-24	2383
February	2023-02-02	1989
March	2023-03-21	5421
March	2023-03-28	2513
March	2023-03-25	2356
April	2023-04-25	3736
April	2023-04-30	2753
April	2023-04-06	2500
May	2023-05-08	8872
May	2023-05-20	6169
May	2023-05-12	6051
June	2023-06-30	8804
June	2023-06-03	8802
June	2023-06-21	7033
July	2023-07-08	3716
July	2023-07-15	3364
July	2023-07-28	2344
August	2023-08-23	2074
August	2023-08-21	1783
August	2023-08-06	1687
September	2023-09-16	3849
September	2023-09-22	3570
September	2023-09-21	2285

```
/* 10. Create a stored procedure that takes the 'Week_no' as input and
generates a report displaying the total shares for each 'Post_type'. The
output of the procedure should consist of two columns:
• post_type
• total_shares */
```

```
DROP PROCEDURE IF EXISTS GetWeeklySharesByPostType;

DELIMITER $$

CREATE PROCEDURE GetWeeklySharesByPostType(IN p_week_no VARCHAR(10))
BEGIN
    SELECT
        fc.post_type,
        SUM(fc.shares) AS total_shares
    FROM fact_content AS fc
    JOIN dim_dates AS dd
        ON fc.date = dd.date
    WHERE dd.week_no = p_week_no
    GROUP BY fc.post_type
    ORDER BY total_shares DESC, fc.post_type;
END $$

DELIMITER ;

CALL GetWeeklySharesByPostType('W1');
```

Result Grid | Filter Rows:

post_type	total_shares
IG Image	1134
IG Reel	659
IG Carousel	40